

Software Design Document

(SDD)

trampoJá

Desenvolvedor: Matheus Barbosa Torres Gomes

Disciplina: Desenvolvimento de Software

Data: Junho/2026

1. Introdução

1.1 Propósito

Este documento descreve o design e a arquitetura do software trampoJá, um marketplace de serviços de curta duração voltado para a cidade de Araxá, MG. O objetivo é documentar as decisões técnicas, a estrutura dos componentes, os fluxos de navegação e as tecnologias utilizadas, servindo como referência para desenvolvimento, manutenção e avaliação do projeto.

1.2 Escopo

O trampoJá conecta clientes que precisam de serviços domésticos e profissionais (jardinagem, encanamento, pintura, eletricista, montagem de móveis, entre outros) com trabalhadores locais de Araxá. O sistema permite que clientes publiquem demandas de serviço com descrição, orçamento e duração, e que trabalhadores interessados se candidatem e conversem diretamente com o cliente.

O projeto possui duas versões paralelas: uma versão web construída com React e Vite para uso em navegadores, e uma versão mobile construída com React Native e Expo para uso no Expo Go em dispositivos Android. Ambas compartilham a mesma lógica de negócio e se conectam ao mesmo backend Firebase.

1.3 Definições e Siglas

SDD - Software Design Document, documento que descreve a arquitetura e design do software.

Firebase Auth: Serviço de autenticação do Google Firebase que gerencia login, registro e sessões de usuários.

Expo Go: Aplicativo que permite rodar projetos React Native diretamente no celular sem necessidade de compilar um build nativo.

Leaflet : Biblioteca JavaScript de código aberto para mapas interativos na web.

OWASP: Open Web Application Security Project, organização que publica diretrizes de segurança para aplicações web.

2. Visão Geral do Sistema

2.1 Descrição Geral

O trampoJá funciona como um marketplace bilateral: de um lado, clientes publicam necessidades de serviço; do outro, trabalhadores navegam essas demandas e se candidatam. A plataforma facilita a descoberta, a negociação e a contratação de serviços de curta duração na região de Araxá.

O fluxo principal do sistema é: o cliente se cadastra, publica um serviço descrevendo o que precisa, quanto pretende pagar e a duração estimada. Trabalhadores interessados visualizam o serviço no feed ou no mapa, se candidatam enviando uma mensagem e proposta de preço, e então cliente e trabalhador conversam diretamente pelo chat integrado para combinar detalhes. Após a conclusão, ambos podem se avaliar mutuamente.

2.2 Categorias de Serviço

O sistema suporta 18 categorias de serviço: Jardinagem, Pedreiro, Encanador, Eletricista, Montador de Móveis, Pintor, Faxineira, Limpador de Estofados, Passeador de Pets, Pet Sitter, Banho e Tosa em Domicílio, Churrasqueiro, Garçom/Copeira, Animador de Festa Infantil, Manicure/Pedicure, Cabeleireiro/Barbeiro, Carreto/Mudanças. Cada categoria possui um ícone visual para identificação rápida no feed e no mapa.

2.3 Usuários do Sistema

O sistema possui dois perfis de usuário, definidos durante o cadastro. O cliente é quem publica demandas de serviço, visualiza candidatos, aceita propostas e avalia o trabalho realizado. O profissional (trabalhador) é quem navega o feed de serviços disponíveis, se candidata aos que lhe interessam, conversa com clientes e recebe avaliações. Na versão atual do protótipo, ambos os perfis têm acesso às mesmas funcionalidades, com a diferenciação planejada para versões futuras.

3. Arquitetura do Sistema

3.1 Padrão Arquitetural

O TrampoJá utiliza uma arquitetura front-end-only com Backend-as-a-Service (BaaS). O Firebase fornece os serviços de backend (autenticação, banco de dados, hosting) enquanto toda a lógica de apresentação e interação reside no frontend React/React Native. Não há servidor backend customizado.

3.2 Camadas da Aplicação

A aplicação é organizada em três camadas lógicas dentro do frontend. A camada de apresentação contém os componentes visuais (telas, cards, botões, modais) responsáveis pela interface do usuário. A camada de lógica de negócio contém a gestão de estado (useState/useEffect do React), validações de formulário, e a lógica de navegação entre telas. A camada de serviços contém a comunicação com o Firebase Auth para autenticação e, futuramente, com o Firestore para persistência de dados.

3.3 Versão Web vs. Versão Mobile

Ambas as versões seguem a mesma estrutura e fluxos, mas utilizam tecnologias diferentes para renderização. A versão web usa React com componentes HTML nativos (div, button, input), CSS inline com variáveis CSS, Leaflet com tiles do Google Maps para o mapa interativo, e Firebase JS SDK (compat) carregado via CDN para autenticação. A versão mobile usa React Native com componentes nativos (View, Text, TouchableOpacity), StyleSheet com objetos JavaScript, react-native-maps com MapView nativo para o mapa, e Firebase JS SDK (modular) instalado via npm.

3.4 Gerenciamento de Estado

O estado da aplicação é gerenciado localmente usando os hooks useState e useEffect do React. O estado principal reside no componente raiz (App/TrampoJa) e inclui: o usuário autenticado, a tela atual, o histórico de navegação para o botão voltar, a lista de serviços, as notificações, e referências ao serviço e trabalhador selecionados. O estado é passado para os componentes filhos via props, seguindo o padrão de prop drilling que é adequado para o tamanho atual da aplicação.

4. Design dos Componentes

4.1 Componentes Compartilhados

O sistema possui componentes reutilizáveis que são usados em múltiplas telas. O componente Avatar renderiza as iniciais do usuário em um círculo com gradiente, aceitando tamanho e cores customizáveis. O componente Card é um container com bordas arredondadas, sombra e efeito hover opcional, usado para serviços, candidatos e reviews. O componente Stars renderiza estrelas interativas ou estáticas para avaliações. O componente Header é o cabeçalho com gradiente escuro usado em todas as telas internas. O componente Badge é o indicador numérico vermelho usado nos ícones de notificação e chat.

4.2 Telas do Sistema

4.2.1 Tela de Login

Ponto de entrada do app. Contém o logo trampoJá, campo de e-mail com validação de formato, campo de senha com toggle mostrar/ocultar e validação de comprimento mínimo, botão de login com Google (versão web usa signInWithPopup, versão mobile usa AuthSession), botão de login com e-mail/senha via Firebase, link para tela de cadastro, e credenciais demo para teste. Exibe mensagens de erro inline nos campos e um banner geral para erros de autenticação.

4.2.2 Tela de Cadastro

Permite criar uma nova conta. Inclui seletor de perfil (cliente ou profissional) com botões visuais, campos de nome, e-mail, senha e confirmação de senha, botão de cadastro com Google, validação completa de todos os campos antes do envio. O cadastro por e-mail/senha usa createUserWithEmailAndPassword do Firebase e atualiza o displayName com updateProfile.

4.2.3 Tela Home (Feed)

Tela principal após o login. Exibe saudação personalizada com o nome do usuário, barra de ícones para mapa, notificações e chat com badges de contagem, scroll horizontal de pills de categorias para filtragem, cards de estatísticas (serviços ativos, fechados hoje, satisfação média), lista de serviços disponíveis com título, localização, duração, valor, número de interessados e badge de urgência, botão flutuante (FAB) para publicar novo serviço, e menu dropdown no avatar com opção de logout.

4.2.4 Tela de Detalhe do Serviço

Mostra informações completas de um serviço selecionado. Contém cabeçalho com ícone da categoria e badge de urgência, cards de valor, duração e número de interessados, descrição completa do serviço com tags de localização e tempo, informações do cliente que publicou, lista de candidatos com nome, avaliação, número de serviços realizados, proposta de preço e mensagem. Cada candidato tem botões de conversar e aceitar, e o nome é clicável para ver o perfil completo. Na parte inferior, o botão "Tenho interesse nesse trampo!" permite se candidatar.

4.2.5 Tela de Chat

Interface de mensagens em tempo real entre cliente e trabalhador. Exibe cabeçalho com avatar, nome e status online do interlocutor, bolhas de mensagem com estilo diferenciado para mensagens enviadas (gradiente escuro, alinhadas à direita) e recebidas (fundo branco, alinhadas à esquerda), timestamps em cada mensagem, indicador "Hoje" no início da conversa, campo de input com botão de enviar que muda de cor quando há texto digitado. O scroll automático vai para a última mensagem ao enviar.

4.2.6 Tela de Publicar Serviço

Formulário para criar uma nova demanda. Inclui seletor visual de categoria com ícones, campo de título, área de texto para descrição detalhada, campos de valor (R\$) e duração lado a lado, campo de localização (bairro), e botão de publicar que só fica ativo quando os campos obrigatórios estão preenchidos. Ao publicar, o serviço é adicionado ao topo do feed e um toast de confirmação aparece.

4.2.7 Tela de Mapa

Visualização geográfica dos serviços disponíveis em Araxá. Na versão web, usa Leaflet com tiles do Google Maps, marcadores customizados coloridos por categoria com ícone e preço, legenda de categorias, animação de pulso nos serviços urgentes, e card popup ao clicar em um marcador. Na versão mobile, usa react-native-maps com MapView nativo, marcadores customizados com View e Text, e flyTo animado ao selecionar um pin. Os bairros mapeados incluem Centro, Bauxita, São Geraldo, Boa Vista, Santa Rita, Jardim Natália, Vila Silvéria e Alvorada, todos com coordenadas reais de latitude e longitude.

4.2.8 Tela de Notificações

Central de notificações do usuário. Exibe lista de notificações de diferentes tipos (novo candidato, nova mensagem, nova avaliação, proposta aceita, lembrete, serviço concluído), cada uma com ícone, título, descrição, tempo relativo e indicador visual de não lida. Inclui botão para marcar todas como lidas e contador de não lidas no cabeçalho.

4.2.9 Perfil do Trabalhador

Tela de perfil de um profissional. Mostra avatar, nome e especialidade no cabeçalho, card de resumo de avaliações com nota média grande, distribuição de estrelas em barras horizontais e total de avaliações, cards de estatísticas (serviços realizados, nota média, tempo de resposta), lista de reviews com nome do avaliador, serviço, data, nota em estrelas e texto do comentário. Na parte inferior, botões de avaliar e conversar.

4.2.10 Modal de Avaliação

Bottom sheet que permite avaliar um profissional. Contém avatar e nome do trabalhador, estrelas interativas de 1 a 5 (tap para selecionar), tags rápidas que mudam conforme a nota (4-5 estrelas mostra tags positivas como Pontual, Caprichoso, Educado; 1-3 estrelas mostra tags construtivas como Atrasou, Pode melhorar), campo de texto livre para comentário, e botão de enviar que exibe a nota selecionada. Ao enviar, uma notificação é adicionada ao feed e um toast de confirmação aparece.

5. Design de Dados

5.1 Modelo de Dados Atual

Na versão atual do protótipo, os dados são mockados diretamente no código JavaScript como arrays de objetos. Cada serviço contém: id, categoria, título, descrição, orçamento, duração, localização (bairro + índice de coordenada), informações do publicador (nome, avatar, rating), tempo de publicação, número de candidatos, flag de urgência e status.

Cada candidato/trabalhador contém: id, nome, avatar (iniciais), rating, número de serviços realizados, especialidade, preço proposto, mensagem de candidatura e array de reviews. Cada review contém: id, nome do avaliador, avatar, nota (1-5), texto, data e tipo de serviço.

As notificações contém: id, tipo (applicant, message, rating, accepted, reminder, completed), título, corpo, tempo relativo, status de leitura e ícone.

5.2 Modelo Futuro com Firestore

Na evolução para produção, os dados serão persistidos no Firebase Firestore com as seguintes coleções: users (uid, nome, email, avatar, tipo, rating, createdAt), services (id, clientId, categoria, título, descrição, budget, duration, location, geopoint, status, createdAt), applications (id, serviceId, workerId, price, message, status, createdAt), chats (id, participantes, lastMessage, updatedAt), messages (id, chatId, senderId, text, createdAt), reviews (id, fromId, toId, serviceId, rating, text, tags, createdAt), e notifications (id, userId, type, title, body, read, createdAt).

6. Design de Autenticação

6.1 Fluxo de Login por E-mail/Senha

O usuário preenche e-mail e senha na tela de login. O frontend valida o formato do e-mail (regex), o comprimento mínimo da senha (6 caracteres) e se os campos não estão vazios. Se a validação passa, chama `signInWithEmailAndPassword` do Firebase Auth. Em caso de sucesso, o Firebase retorna o objeto user com uid, email, displayName e photoURL. O app extrai as iniciais do nome para o avatar e navega para a tela Home. Em caso de erro, a mensagem é traduzida para português e exibida ao usuário.

6.2 Fluxo de Login com Google

Na versão web, o botão Google chama `signInWithPopup` com `GoogleAuthProvider`, que abre uma janela popup do Google para o usuário selecionar sua conta. Após a autorização, o Firebase retorna o user autenticado. Na versão mobile (Expo), o fluxo usa `AuthSession` com o discovery document do Google, solicitando um `id_token` que é então trocado por uma credencial Firebase via `GoogleAuthProvider.credential` e `signInWithCredential`.

6.3 Fluxo de Registro

O usuário seleciona seu perfil (cliente ou profissional), preenche nome, e-mail, senha e confirmação. O frontend valida todos os campos incluindo se as senhas conferem. Chama `createUserWithEmailAndPassword` e em seguida `updateProfile` para salvar o displayName. O usuário é automaticamente logado após o registro.

6.4 Fluxo de Logout

O botão de logout no menu do avatar chama `signOut` do Firebase Auth, limpa o estado do React (user, tela, histórico, seleções) e redireciona para a tela de login.

7. Design de Navegação

7.1 Sistema de Navegação

A navegação entre telas é gerenciada por um sistema de estado simples com histórico. O estado screen determina qual tela está visível, e o array history mantém o histórico de telas visitadas para permitir navegação com o botão voltar. A função nav(screen) empilha a tela atual no histórico e muda para a nova tela. A função back() desempilha a última tela do histórico e retorna a ela.

7.2 Mapa de Navegação

O fluxo principal de navegação é: Login permite ir para Home (após autenticação) ou para Register. Register permite voltar para Login ou ir para Home (após cadastro). Home é o hub central que permite acessar Detail (ao clicar em serviço), Post (pelo FAB), Chats (pelo ícone de chat), Notifications (pelo ícone de sino), Map (pelo ícone de pin), e Logout (pelo menu do avatar).

A partir de Detail, o usuário pode ir para Chat (ao conversar com candidato) ou Worker (ao clicar no nome do candidato). A partir de Worker, pode ir para Chat (botão conversar) ou abrir o Rating Modal (botão avaliar). Todas as telas internas possuem botão Voltar que retorna à tela anterior no histórico.

8. Stack Tecnológico

8.1 Frontend Web

React 18+ como framework de UI, Vite como bundler e servidor de desenvolvimento, CSS inline com variáveis CSS para tematização, Leaflet 1.9.4 com tiles do Google Maps para mapas interativos, Firebase JS SDK 10.12.0 (compat) carregado via CDN para autenticação, Google Fonts (Archivo Black e DM Sans) para tipografia.

8.2 Frontend Mobile

React Native com Expo SDK para desenvolvimento cross-platform, expo-linear-gradient para gradientes nativos, react-native-maps para mapas nativos com MapView, Firebase JS SDK (modular) instalado via npm, expo-auth-session para fluxo OAuth do Google Sign-In.

8.3 Backend (BaaS)

Firebase Authentication para gestão de usuários, login e sessões. Firebase Hosting para deploy da versão web. Firestore planejado para persistência de dados em produção.

8.4 Testes

Playwright para testes end-to-end automatizados da versão web, com 14 casos de teste cobrindo validação de formulários, navegação entre telas, autenticação e funcionalidades principais. Screenshots são gerados automaticamente como evidência.

8.5 Segurança

Análise baseada no padrão OWASP Top 10 (2021) documentada em relatório separado, cobrindo controle de acesso, criptografia, injeção, design seguro, configuração, componentes, autenticação, integridade, logging e SSRF.

9. Considerações e Evolução

9.1 Limitações Atuais

O protótipo atual utiliza dados mockados no frontend, sem persistência real. A diferenciação de permissões entre perfis de cliente e profissional ainda não está implementada. O Google Sign-In na versão Expo Go apresenta incompatibilidades com o proxy de autenticação deprecado nas versões recentes do SDK. A versão web não pode ser executada no modo web do Expo devido à incompatibilidade do react-native-maps com o bundler webpack.

9.2 Roadmap de Evolução

As próximas etapas planejadas para o projeto incluem: integração com Firebase Firestore para persistência de dados em tempo real, implementação de notificações push com Firebase Cloud Messaging, sistema de pagamento integrado para transações entre cliente e trabalhador, geolocalização em tempo real para serviços próximos, sistema de verificação de identidade para profissionais, e publicação nas lojas (Google Play e App Store) com build nativo via EAS Build.